



SESSION 3

MODULES AND PACKAGES IN NODE.JS

Mục tiêu học tập

Trong buổi học này, học sinh sẽ học cách:

- Giải thích mô-đun là gì và ba loại của nó
- Giải thích ý nghĩa của các gói
- Mô tả mục đích của khung Web và các chức năng tiện ích

Mô-đun là tập hợp các hàm hoặc phương thức đã được kiểm nghiệm theo thời gian, có thể tái sử dụng và sẵn sàng sử dụng, có thể được các ứng dụng khác sử dụng để giúp quá trình phát triển dễ dàng hơn.

Gói là tập hợp một hoặc nhiều mô-đun được nhóm lại thành một đơn vị duy nhất có thể được xuất bản và cài đặt trong các ứng dụng Node.js. Khung web là tập hợp các thư viện và công cụ cung cấp các tính năng cụ thể.

Sử dụng các khung Web trong các ứng dụng Node.js giúp đơn giản hóa quá trình phát triển và giảm nhu cầu phải lặp lại cùng một mã ở nhiều nơi.

Buổi học này sẽ giải thích các loại mô-đun và khung web khác nhau có thể được sử dụng để xây dựng các ứng dụng Node.js. Buổi học cũng sẽ giải thích cách cài đặt và sử dụng các gói. Cuối cùng, buổi học sẽ thảo luận về tầm quan trọng của các hàm tiện ích.

3.1 Các mô-đun trong Node.js

Mô-đun có thể được coi là một ứng dụng, trong đó các hàm hoặc phương thức được gói gọn lại với nhau. Chúng có sẵn dưới dạng ứng dụng tức thời, sẵn sàng sử dụng. Mô-đun phải được nhập vào mã để sử dụng các hàm được định nghĩa trong đó. Ưu điểm của mô-đun là sau khi được định nghĩa, chúng có thể được sử dụng lại bên trong bất kỳ số lượng tệp chương trình Node.js nào.



Phương thức cũng là một hàm được liên kết với một đối tượng. Tuy nhiên, một hàm không được liên kết với bất kỳ đối tượng nào. Ví dụ, phương thức GET có thể được định nghĩa cho một đối tượng HTTP Request hoặc HTTP Response. Require là một hàm Node.js được sử dụng để import các module trong chương trình Node.js.

Sau đây là các loại mô-đun khác nhau trong Node.js:



3.1.1 Mô-đun cốt lõi

Các mô-đun tích hợp được gọi là các mô-đun cốt lõi, là một phần của Node.js nền tảng. Các mô-đun tích hợp được cài đặt cùng với quá trình cài đặt Node.js.

Bảng 3.1 liệt kê một số mô-đun cốt lõi.

Tên của Mô-đun	Sự miêu tả
http	Cung cấp các phương pháp để tạo máy chủ HTTP trong Node.js.
khẳng định	Cung cấp các phương pháp với một tập hợp các hàm khẳng định để thử nghiệm.
fs	Cung cấp các phương pháp để xử lý hệ thống tập tin.
con đường	Cung cấp các phương pháp xử lý đường dẫn tệp.
quá trình	Cung cấp các phương pháp để lấy thông tin và kiểm soát tiến trình Node.js hiện tại.
tự điều hành	Cung cấp các phương pháp để lấy thông tin về hệ điều hành.
chuỗi truy vấn	Cung cấp các phương pháp để phân tích cú pháp và định dạng chuỗi truy vấn URL.
địa chỉ	Cung cấp các phương pháp phân tích và giải quyết URL.

Bảng 3.1: Các mô-đun tích hợp

3.1.2 Mô-đun cục bộ

Mô-đun cục bộ là một tệp .js chứa các hàm do người dùng định nghĩa. Mô-đun này được tạo cục bộ trong ứng dụng Node.js và có thể được phân phối trên nhiều dự án bằng Trình quản lý Gói Node (NPM). Ví dụ: một mô-đun cục bộ có thể được tạo để chứa một hàm trả về ngày và giờ hiện tại và có thể được phân phối trên nhiều dự án.

3.1.3 Mô-đun của bên thứ ba

Mô-đun của bên thứ ba là các mô-đun không phải là mô-đun gốc của Node.js. Chúng được tạo bởi một bên thứ ba để sử dụng cho các ứng dụng web khác và có sẵn trực tuyến. Chúng có thể được cài đặt bằng NPM.

Bảng 3.2 liệt kê một số mô-đun của bên thứ ba.

Tên của mô-đun	Sự miêu tả
cây mangut	Đây là thư viện mô hình hóa dữ liệu đối tượng MongoDB. Nó chuyển đổi mã từ MongoDB sang máy chủ Node.js.
Thẻ hiện	Đây là thư viện dùng để xây dựng ứng dụng web và di động. Nó quản lý việc định tuyến của máy chủ.
Góc	Đây là một thư viện dùng để xây dựng các ứng dụng web mạnh mẽ, dựa trên mô hình MVC.
Phản ứng	Đây là một thư viện dùng để xây dựng giao diện front-end của ứng dụng. Nó hỗ trợ tạo ra các giao diện người dùng sáng tạo và phức tạp.

Bảng 3.2: Các mô-đun của bên thứ ba

Một lợi thế của việc sử dụng mô-đun của bên thứ ba là không tốn chi phí viết mã và kiểm thử. Mã đã được kiểm thử và có thể sử dụng ngay lập tức.

3.2 Làm việc với các Mô-đun

Node.js cung cấp hàm require để nhập các mô-đun. Cú pháp cho sử dụng hàm require như sau:

```
Var_module_name = yêu cầu('module_name')
```

Tại đây, Var_module_name là tên của biến mà phiên bản của mô-đun được nhập được gán vào. module_name là tên của mô-đun đang được nhập như OS, http và fs.

Đoạn mã 1 hiển thị một mã ví dụ nhập mô-đun tích hợp fs sử dụng hàm require và gán nó cho biến có tên Var_fs. Sau đó, mã sẽ in một thông báo ra bảng điều khiển.

Đoạn mã 1:

```
var_fs=yêu cầu('fs');
console.log('Một phiên bản của đối tượng Hệ thống tệp đã được tạo');
```

Để thực thi mã, hãy nhập mã vào VS Code và lưu với tên tệp là code-snippet1.js. Sau đó, thực thi tệp .js bằng lệnh như sau:

```
đoạn mã nút-snippet1.js
```

Hình 3.1 hiển thị kết quả thực thi của Đoạn mã 1.

```
C:\Users\write\Sample code>node code-snippet1.js
An instance of File system object created
```

Hình 3.1: Thực thi code-snippet1.js

Sau khi nhập mô-đun vào mã Node.js, hãy bắt đầu sử dụng các phương thức trong mô-đun bằng cách truy cập chúng thông qua thể hiện của mô-đun. Ví dụ: mô-đun fs cung cấp một tập hợp các phương thức để thực hiện các hành động được chỉ định trên một tệp:

- Mở một tập tin
- Ghi vào một tập tin
- Thêm một tập tin
- Đọc một tập tin
- Xóa một tập tin

Bảng 3.3 mô tả các phương pháp này.

Phương pháp	Sự miêu tả
fs.open	Thực hiện thao tác được chỉ định trên tệp được chỉ định, chẳng hạn như đọc, ghi và thêm. Nếu tệp không tồn tại, sẽ tạo một tệp mới. Mở tệp bằng thao tác được đề cập, chẳng hạn như đọc hoặc ghi hoặc đọc và ghi. Tạo tệp nếu tệp không tồn tại.
fs.writeFile	Thực hiện thao tác ghi vào tệp. Tạo tệp mới nếu tệp chưa tồn tại. Ghi đè lên tệp nếu tệp đã tồn tại.
fs.appendFile	Thêm nội dung đã cho vào tệp nếu tệp đó đã tồn tại. Nếu không, hãy tạo một tệp mới và thêm nội dung vào đó.
fs.readFile	Đọc nội dung của một tập tin.
fs.unlink	Xóa tệp khỏi thư mục.

Bảng 3.3: Phương pháp xử lý tệp

3.2.1 Mở một tập tin

Cú pháp để mở hoặc tạo tệp mới bằng phương thức `open` như sau:

```
fs.open(đường dẫn, cờ[,chế độ], gọi lại)
```

Đây:

- `path` biểu thị đường dẫn đến tệp cần mở.
- `flags` biểu thị thao tác mà tệp sẽ được mở. Các cờ có thể được sử dụng để mở tệp cho thao tác đọc hoặc ghi.
- `mode` chỉ định chế độ của tệp mà tệp phải được mở.
- `callback` là hàm gọi lại được gọi sau khi thao tác mở tệp hoàn tất.

Đoạn mã 2 hiển thị mã để tạo tệp bằng phương thức `fs.open`.

Mã này tạo một tệp văn bản trống có tên `filecontent.txt` ở chế độ ghi.

Đoạn mã 2:

```
var fs = require('fs');
fs.open('filecontent.txt', 'w', hàm (err) {

    nếu (err) ném err;
    console.log("Tệp đã được tạo thành công.")
});
```

Trong Đoạn mã 2, `function(err)` là tham số của phương thức `fs.open`.

Đây là hàm gọi lại được gọi nếu xảy ra lỗi như `err`

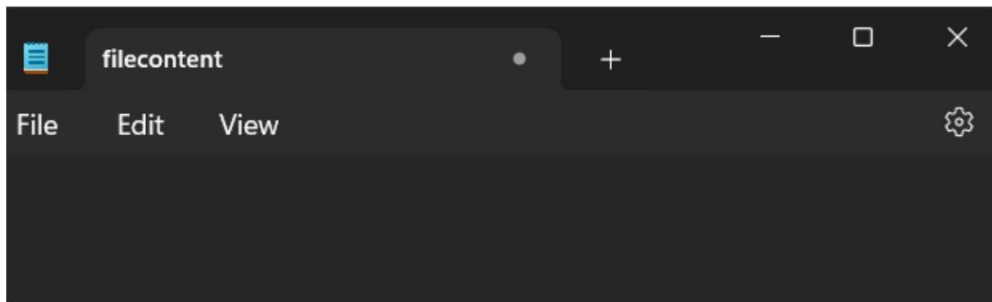
Tham số được chỉ định. Hàm callback này sẽ trả về lỗi được chỉ định bởi tham số `err`. Nếu không có lỗi nào xảy ra khi mở tệp, thông báo "Tệp đã được tạo thành công" sẽ được ghi vào bảng điều khiển. Nhập mã vào VS Code và lưu tệp dưới dạng `openfile.js`.

Hình 3.2 hiển thị kết quả thực thi của tệp.

```
C:\Users\write\Sample code>node openfile.js
File created successfully.
```

Hình 3.2: Thực thi `openfile.js`

Hình 3.3 hiển thị nội dung của `filecontent.txt`, được mở bằng Notepad.



Hình 3.3: Nội dung của tệp filecontent.txt

3.2.2 Ghi vào tệp

Cú pháp để tạo tệp bằng phương thức writeFile là:

```
fs.writeFile(tên tệp, dữ liệu, [mã hóa], [hàm gọi lại])
```

Đây:

- filename biểu thị tên tệp.
- data là nội dung được ghi vào tệp.
- encoding là tham số tùy chọn biểu diễn cách dữ liệu được thể hiện bên trong hệ thống tệp. ASCII, Base64, UTF64 là các định dạng mã hóa phổ biến. UTF64 là định dạng mã hóa thường được sử dụng để biểu diễn tệp văn bản nội bộ.
- callback_function là tham số tùy chọn chỉ định hàm callback, được gọi sau khi nội dung được ghi vào tệp. Hàm này có hai tham số:

- o err: Là một đối tượng lỗi chứa thông báo lỗi nếu có bất kỳ lỗi nào xảy ra.

- o dữ liệu: Là bộ đệm hoặc chuỗi chứa nội dung tệp.

Đoạn mã 3 hiển thị mã để tạo tệp có tên nodefile.txt và viết thông báo tùy chỉnh vào đó.

Đoạn mã 3:

```
var fs = yêu cầu('fs');
fs.writeFile('nodefile.txt', 'Chào mừng đến với Lập trình Node.js. Trong buổi học này, chúng ta sẽ tìm hiểu về mô-đun Hệ thống tệp.', function (err) {

    nếu (err) ném err;
    console.log('Tập tin đã được ghi thành công.');
```

Trong Đoạn mã 3, tên tệp, dữ liệu cần ghi và hàm callback được truyền dưới dạng tham số cho phương thức `writeFile`. Nếu xảy ra lỗi, hàm callback được gọi bằng cách truyền mã lỗi `err`. Nếu phương thức thực thi thành công, một thông báo sẽ được hiển thị trên bảng điều khiển.

Nhập mã vào VS Code và lưu với tên tệp là `creatfile.js`.

Thực thi mã. Hình 3.4 hiển thị kết quả thực thi của đoạn mã 3.

```
C:\Users\write\Sample code>node creatfile.js
File is written successfully.
```

Hình 3.4: Thực thi `creatfile.js`

Để xác minh tệp văn bản đã được tạo, hãy nhập lệnh `dir *.txt` tại dấu nhắc lệnh. Tìm tệp được liệt kê trong thư mục hiện tại.

Hình 3.5 hiển thị tệp `nodefile.txt` được liệt kê trong thư mục hiện tại.

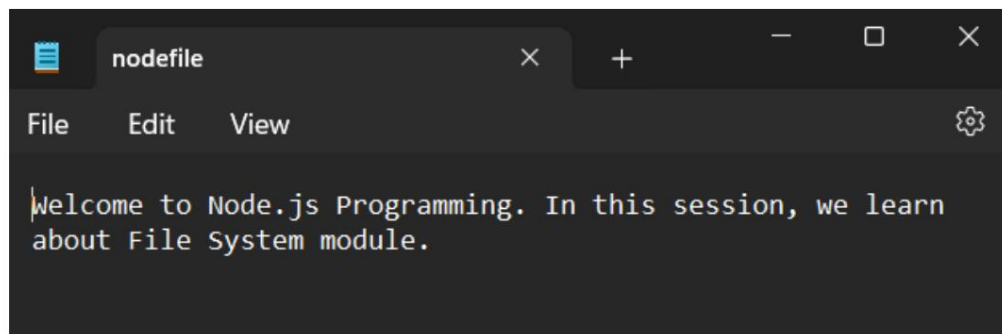
```
C:\Users\write\Sample code>dir *.txt
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write\Sample code

09-11-2023  15:16                0 filecontent.txt
03-11-2023  07:33            136 in the post..txt
09-11-2023  16:22            83 nodefile.txt
13-10-2023  05:19                0 openfile.txt
13-10-2023  11:34           160 readfile.js.txt
13-10-2023  15:09            42 sampleopen.txt
13-10-2023  11:44            93 sampleread.txt
13-10-2023  04:25            51 samplewrite.txt
               8 File(s)          565 bytes
               0 Dir(s)  74,353,127,424 bytes free
```

Hình 3.5: Tệp được liệt kê trong thư mục

Hình 3.6 hiển thị nội dung của tệp `nodefile.txt`.



Hình 3.6: Nội dung của tệp `nodefile.txt`

3.2.3 Thêm vào một tệp

Thêm là thêm nội dung vào cuối tệp. Phương thức `appendFile` thêm dữ liệu đã cho vào cuối tệp. Nếu tệp không tồn tại, nó sẽ tạo một tệp mới.

Cú pháp để thêm tệp bằng phương thức `appendFile` là:

```
fs.appendFile(tên tệp, dữ liệu, [tùy chọn], gọi lại)
```

Đây:

- `filename` biểu thị tên của tệp.
- `data` là nội dung được ghi vào tệp.
- `options` là tham số tùy chọn biểu thị mã hóa, chế độ hoặc lá cờ.
- `callback_function` là một tham số tùy chọn chỉ định hàm callback, được gọi sau khi nội dung được ghi vào tệp. Hàm này có hai tham số:

- o `err`: Là một đối tượng lỗi chứa thông báo lỗi nếu có bất kỳ lỗi nào xảy ra.

- o `dữ liệu`: Là bộ đệm hoặc chuỗi chứa nội dung tệp.

Đoạn mã 4 hiển thị mã để thêm tệp.

Đoạn mã 4:

```
var fs = yêu cầu('fs');
var data="ở đây chúng ta tạo, đọc và ghi nội dung vào tệp." fs.appendFile('nodefile.txt', data, function
(err) {

    nếu (err) ném err;
    console.log("Đã thêm tệp thành công.")
});
fs.readFile('nodefile.txt', hàm(err, dữ liệu)
{
    nếu (err) ném err;
    console.log(data.toString('utf8'))
});
```

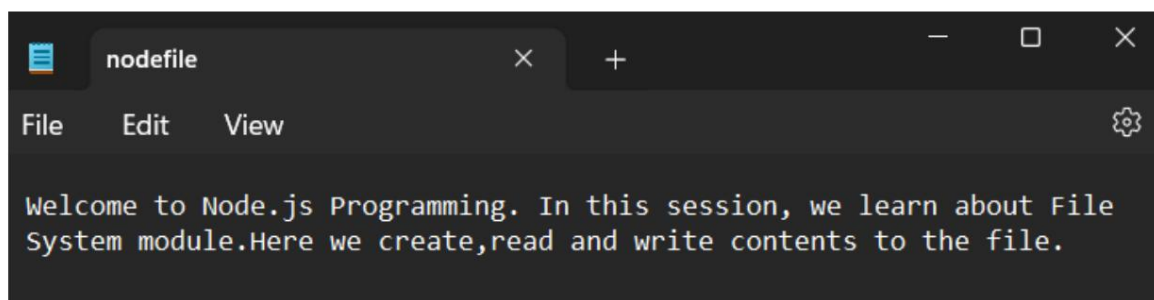
Trong Đoạn mã 4, phương thức `appendFile` sẽ thêm nội dung chuỗi được lưu trữ trong biến có tên `data` vào tệp `nodefile.txt`. Sau đó, phương thức `readFile` sẽ đọc và hiển thị nội dung của tệp `nodefile.txt` ra bảng điều khiển.

Lưu mã vào tệp có tên `appendfile.js` và thực thi tệp. Hình 3.7 hiển thị kết quả thực thi. Kết quả hiển thị nội dung của tệp `nodefile.txt` với nội dung được thêm vào.

```
C:\Users\write\Sample code>node appendfile.js
Welcome to Node.js Programming. In this session, we learn about File System module.Here we
create,read and write contents to the file.
File Appended successfully.
```

Hình 3.7: Thực thi `appendfile.js`

Hình 3.8 hiển thị nội dung của tệp văn bản `nodefile.txt` nơi dữ liệu văn bản được thêm vào.



The screenshot shows a text editor window with the title 'nodefile'. The menu bar includes 'File', 'Edit', and 'View'. The text content of the file is: 'Welcome to Node.js Programming. In this session, we learn about File System module.Here we create,read and write contents to the file.'

Hình 3.8: Nội dung của tệp `nodefile.txt`

3.2.4 Đọc một tập tin

Để đọc nội dung của các tệp văn bản, hãy sử dụng phương thức `readFile` của `fs` mô-đun.

Cú pháp của phương thức `readFile` là:

```
fs.readFile(tên tệp, lệnh gọi lại)
```

Tại đây, `filename` chỉ định đường dẫn đến tệp cần đọc. `callback` là hàm callback được gọi sau khi thao tác đọc hoàn tất.

Đoạn mã 5 hiển thị mã để đọc tệp.

Đoạn mã 5:

```
var fs = require('fs');
fs.readFile('nodefile.txt', hàm (err) {

    nếu (err) ném err;
    console.log("Đã đọc tệp thành công.")
});
```

Mã này đọc một tệp văn bản có tên là `nodefile.txt`.

Để thực thi mã, hãy lưu mã với tên tệp là `readfile.js` và thực thi tệp. Hình 3.9 hiển thị kết quả thực thi. Nội dung của tệp `nodefile.txt` được hiển thị trong bảng điều khiển.

```
C:\Users\write\Sample code>node readfile.js
Welcome to Node.js Programming. In this session, we learn about File System module. Here we
create, read and write contents to the file.
```

Hình 3.9: Thực thi `readfile.js`

3.2.5 Xóa một tập tin

Các phương thức `fs.unlink` và `fs.rmdir` giúp xóa một tập tin.

Cú pháp để xóa tệp bằng phương thức `unlink` là:

```
fs.unlink(tên tệp, lệnh gọi lại)
```

Tại đây, `filename` biểu thị đường dẫn đến tệp cần xóa. `callback` là tham số tùy chọn chỉ định hàm sẽ được gọi sau khi thao tác xóa hoàn tất.

Đoạn mã 6 hiển thị mã để xóa tệp.

Đoạn mã 6:

```
var fs = require("fs");
fs.unlink('filecontent.txt', hàm(err) {
  nếu (err) ném err;
  console.log("Đã xóa tệp thành công!");
});
```

Mã này gọi phương thức `unlink` để hủy liên kết tệp đã chỉ định khỏi thư mục hiện tại, do đó xóa tệp đó khỏi thư mục.

Lưu mã với tên tệp là `deletfile.js`. Chạy tệp. Tệp `filecontent.txt` được sử dụng làm tệp mẫu cho quá trình xóa. Sau khi mã thực thi, tệp sẽ bị xóa. Hình 3.10 minh họa quá trình thực thi Đoạn mã 6.

```
C:\Users\write\Sample code>node deletfile.js
File deleted successfully!
```

Hình 3.10: Thực thi `deletfile.js`

Hình 3.11 cho thấy quá trình thực thi lệnh `dir *.txt` để hiển thị danh sách tệp `filecontent.txt` trong thư mục hiện tại trước và sau khi thực thi mã trong `deletefile.js`.

```
C:\Users\write\Sample code>dir *.txt
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write\Sample code

09-11-2023  16:41                0 filecontent.txt
03-11-2023  07:33            136 in the post..txt
09-11-2023  18:21            134 nodefile.txt
13-10-2023  05:19                0 openfile.txt
13-10-2023  11:34            160 readfile.js.txt
09-11-2023  17:42            195 sampleopen.txt
13-10-2023  11:44             93 sampleread.txt
13-10-2023  04:25             51 samplewrite.txt
               8 File(s)              769 bytes
               0 Dir(s)  74,326,167,552 bytes free

C:\Users\write\Sample code>node deletefile.js
File deleted successfully!

C:\Users\write\Sample code>dir *.txt
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write\Sample code

03-11-2023  07:33            136 in the post..txt
09-11-2023  18:21            134 nodefile.txt
13-10-2023  05:19                0 openfile.txt
13-10-2023  11:34            160 readfile.js.txt
09-11-2023  17:42            195 sampleopen.txt
13-10-2023  11:44             93 sampleread.txt
13-10-2023  04:25             51 samplewrite.txt
               7 File(s)              769 bytes
               0 Dir(s)  74,327,367,680 bytes free
```

Hình 3.11: Danh sách thư mục

3.3 Gói

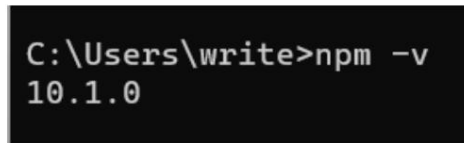
Gói là một tập hợp các mô-đun. Gói nhóm tất cả các mô-đun liên quan lại thành một đơn vị duy nhất để có thể nhập vào ứng dụng.

Để nhập một gói vào ứng dụng, hãy sử dụng hàm `require`. Các hàm trong các mô-đun của một gói có thể được truy cập bằng cách sử dụng thể hiện được trả về bởi hàm `require`.

3.3.1 Cài đặt các gói

Để cài đặt các gói, hãy sử dụng công cụ dòng lệnh npm. Công cụ dòng lệnh này được cài đặt cùng với Node.js. Để xác minh việc cài đặt npm, hãy nhập lệnh `npm -v` tại dấu nhắc lệnh.

Hình 3.12 hiển thị kết quả xác minh phiên bản của công cụ npm.



```
C:\Users\write>npm -v
10.1.0
```

Hình 3.12: Xác minh phiên bản của npm

Để cập nhật phiên bản npm cũ hơn lên phiên bản mới nhất, hãy sử dụng lệnh như sau:



```
npm cài đặt npm -g
```

Tại

đây, • npm là tên của công cụ dòng lệnh chạy lệnh. • install là lệnh cài đặt gói được chỉ định.
• g là cờ biểu thị rằng npm phải được cài đặt toàn cục trên hệ thống, thay vì cài đặt cục bộ trong thư mục dự án hiện tại.

Cài đặt gói cục bộ

Để cài đặt các gói cục bộ trong thư mục dự án hiện tại, hãy sử dụng lệnh như sau:



```
npm install <tên_gói>
```

Ở đây, `package_name` là tên gói cần cài đặt. Ví dụ: lệnh để cài đặt Angular CLI cục bộ trong thư mục dự án hiện tại của bạn là:



```
npm cài đặt @angular/cli
```

Hình 3.13 hiển thị quá trình thực hiện lệnh này tại dấu nhắc lệnh và cài đặt các gói Angular.

```
C:\Users\write>npm install @angular/cli
added 244 packages in 1m
```

Hình 3.13: Cài đặt gói 'angular'

Các gói được cài đặt cục bộ chỉ khả dụng cho các ứng dụng Node.js nằm trong thư mục cục bộ tương ứng. Theo mặc định, NPM sẽ đặt các mô-đun của một gói vào thư mục `node_modules` cục bộ. Để xác minh, hãy nhập `dir` tại dấu nhắc lệnh. Tìm thư mục `node_modules` được liệt kê như trong Hình 3.14.

```
C:\Users\write>dir
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write

09-11-2023  18:39    <DIR>          .
06-03-2023  20:28    <DIR>          ..
09-05-2021  13:16    <DIR>          .android
14-09-2021  11:25    <DIR>          .fcc
19-10-2023  10:55    <DIR>          .ipynb_checkpoints
28-06-2023  19:28    <DIR>          .ipython
07-08-2023  09:40    <DIR>          .jupyter
03-09-2023  10:26    <DIR>          .matplotlib
21-09-2023  12:49          13 .node_repl_history
27-09-2023  11:25    <DIR>          .vscode
01-12-2020  18:39    <DIR>          3D Objects
02-12-2020  13:18    <DIR>          ansel
09-11-2023  17:25          381 appendfile.js
19-10-2023  10:53          3,698 code.txt
06-03-2023  20:39    <DIR>          Contacts
07-08-2023  11:52          366 create_table.py
09-11-2023  18:38          151 deeltefile.js
09-11-2023  18:40          151 deletefile.js
03-04-2023  18:09    <DIR>          Documents
09-11-2023  12:02    <DIR>          Downloads
06-03-2023  20:39    <DIR>          Favorites
28-08-2023  19:18          417 filextest.pkl
28-08-2023  19:18          945 filextrain.pkl
28-08-2023  19:18          241 fileytest.pkl
28-08-2023  19:18          285 fileytrain.pkl
19-07-2023  15:46    <DIR>          flask_app
17-06-2023  17:09          2,578,580 get-pip.py
06-03-2023  20:39    <DIR>          Links
06-03-2023  20:39    <DIR>          Music
09-11-2023  20:08    <DIR>          node_modules
09-11-2023  05:56    <DIR>          OneDrive
09-11-2023  20:08          128,278 package-lock.json
09-11-2023  20:08          187 package.json
```

Hình 3.14: Thư mục `node_modules`

Cài đặt gói toàn cầu

Để cho phép các ứng dụng Node.js hiện diện trên toàn hệ thống truy cập một gói, hãy cài đặt gói đó trên toàn cục bằng cách bật chế độ toàn cục. Thêm -g vào lệnh npm install. Ví dụ: để cài đặt Angular CLI trên toàn cục trên hệ thống, hãy sử dụng lệnh như sau:

```
npm cài đặt -g @angular/cli
```

3.3.2 Cập nhật các gói

Để cập nhật gói, hãy sử dụng lệnh npm update như sau:

3.3.3 Gỡ cài đặt gói

```
npm update <tên gói>
```

Để gỡ cài đặt một gói, hãy sử dụng lệnh npm uninstall như sau:

```
npm uninstall <tên gói>
```

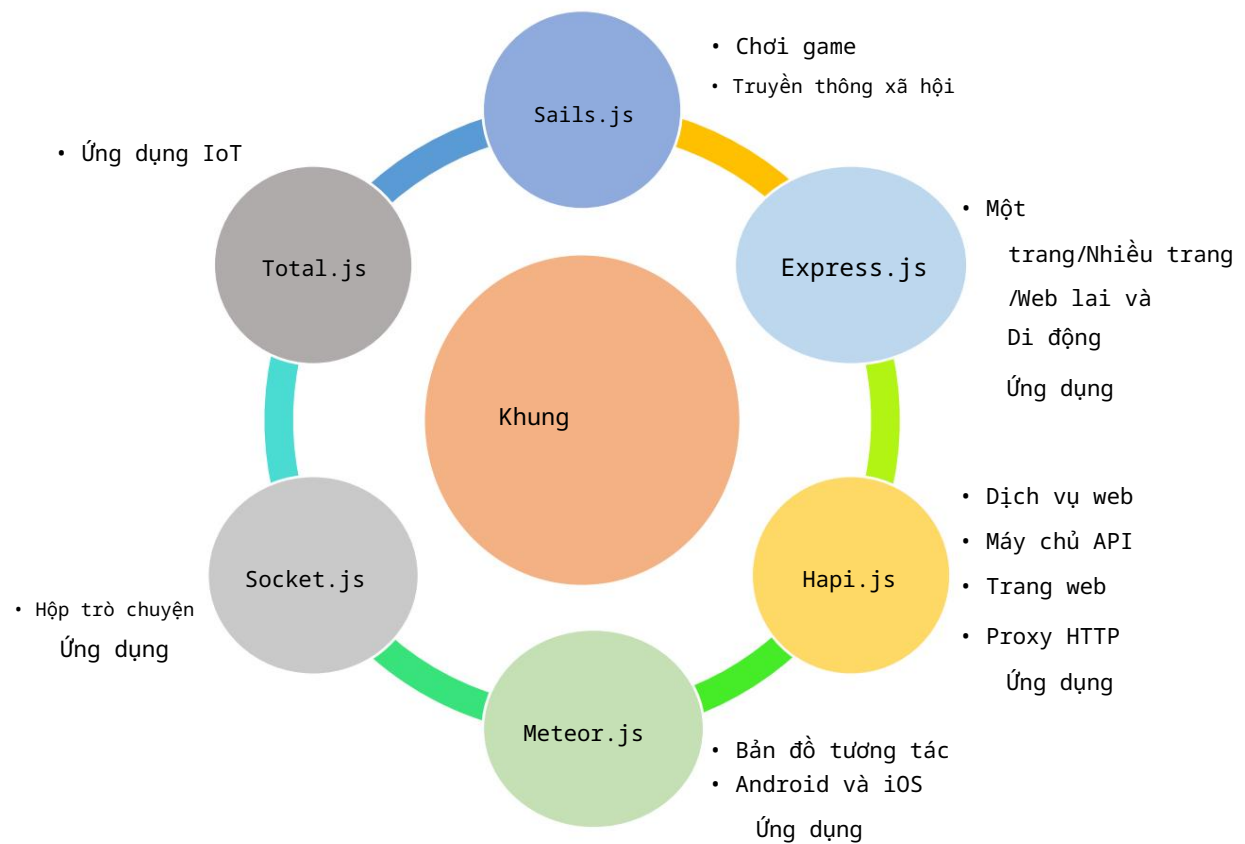
Hình 3.15 hiển thị quá trình thực hiện lệnh này tại dấu nhắc lệnh và gỡ cài đặt các gói Angular.

```
C:\Users\write>npm uninstall @angular/cli  
  
removed 244 packages, and audited 1 package in 1s  
  
found 0 vulnerabilities
```

Hình 3.15: Gỡ cài đặt gói

3.4 Khung web

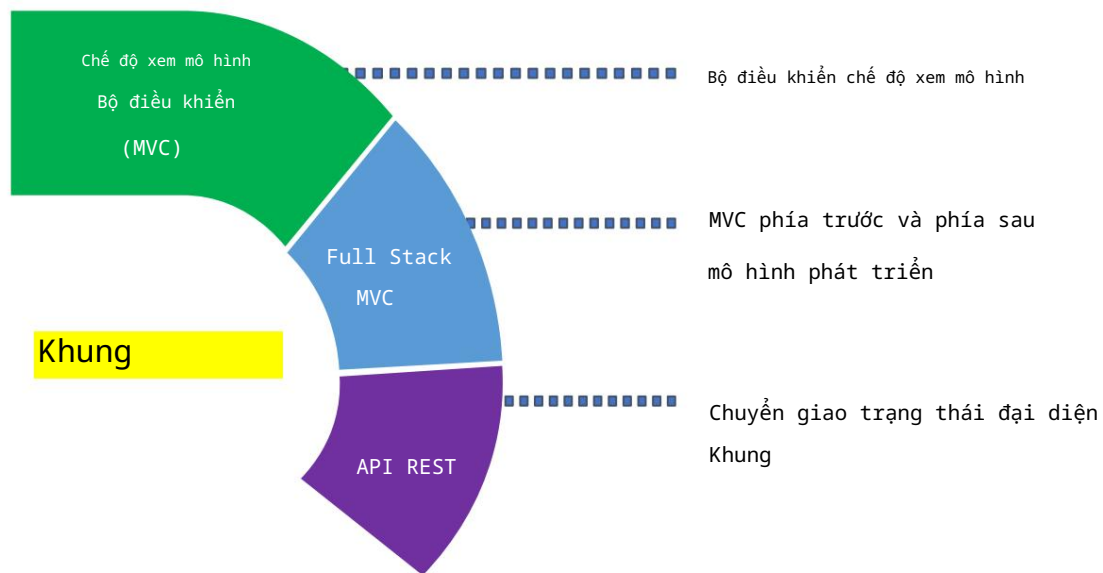
Khung web là một tập hợp các thư viện hoặc công cụ được sử dụng để phát triển nhanh chóng các ứng dụng web. Chúng loại bỏ yêu cầu phải viết mã lặp đi lặp lại. Các framework hoạt động như một khuôn mẫu để thiết kế Giao diện người dùng (UI) của các ứng dụng web. Chúng cũng cung cấp các thư viện để truy cập cơ sở dữ liệu, quản lý phiên và các tác vụ phổ biến khác. Hình 3.16 minh họa một số framework được sử dụng với Node.js.



Hình 3.16: Các khung của Node.js

Node framework là một nền tảng web hỗ trợ phát triển ứng dụng bằng Node.js. Node framework cho phép các nhà phát triển sử dụng JavaScript để phát triển cả front-end và back-end của một ứng dụng web.

Có ba loại mô hình khung trong Node.js. Chúng được mô tả như trong Hình 3.17.

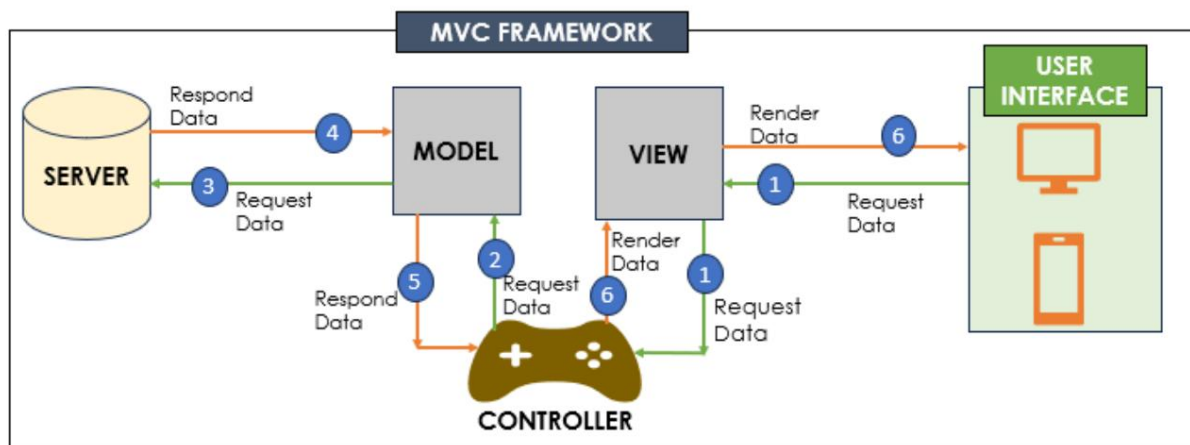


Hình 3.17: Các loại mô hình khung

3.4.1 Khung MVC

MVC là viết tắt của thuật ngữ Model-View-Controller. Mô hình này chia logic mã thành ba phần: Model, View và Controller. Thành phần Model chứa dữ liệu và logic nghiệp vụ. Thành phần View chứa logic để xử lý giao diện người dùng và đầu ra. Thành phần Controller chứa logic để xử lý dữ liệu người dùng nhập vào và phối hợp với Model và View.

Express.js được mô phỏng theo mô hình MVC của Express.js. Hình 3.18 minh họa cách thức hoạt động của mô hình MVC.



Hình 3.18: Khung MVC

Trong mô hình MVC Framework:

1. Người dùng gửi yêu cầu đến Bộ điều khiển thông qua View.
2. Bộ điều khiển lần lượt gửi các yêu cầu đến Mô hình.
3. Mô hình yêu cầu máy chủ cơ sở dữ liệu cung cấp dữ liệu cần thiết.
4. Máy chủ cơ sở dữ liệu gửi dữ liệu cần thiết dưới dạng phản hồi tới Mô hình.
5. Model gửi phản hồi tới Controller.
6. Bộ điều khiển lần lượt sử dụng Chế độ xem để hiển thị phản hồi.

Do đó, Controller đóng vai trò như cầu nối giữa Model và View, kiểm soát luồng thông tin.

3.4.2 Khung MVC đầy đủ

Mô hình MVC Framework đầy đủ hỗ trợ cả phát triển ứng dụng front-end và back-end. Nó cung cấp các thư viện và công cụ chuyên biệt để phát triển và quản lý ứng dụng.

3.4.3 Khung API REST

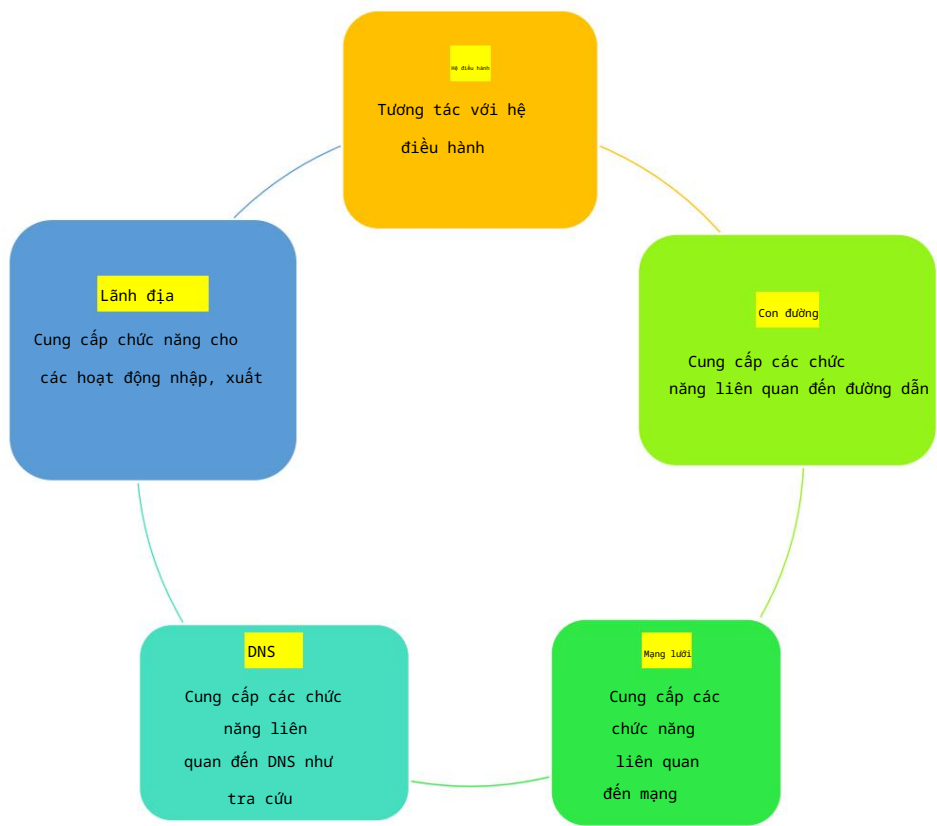
REST là viết tắt của Representational State Transfer (Chuyển giao Trạng thái Biểu diễn). REST API định nghĩa một tập hợp các quy tắc giao tiếp giữa máy chủ và máy khách bằng phương thức HTTP.

Bằng cách sử dụng REST API, các nhà phát triển có thể tạo và sử dụng các dịch vụ web REST trong các ứng dụng Node.js.

Sự đơn giản và dễ sử dụng giúp REST dễ tiếp cận với nhiều nhà phát triển. Hơn nữa, các nhà phát triển đã quen thuộc với các phương thức HTTP như get, put, post và delete, vốn là một phần của REST API.

3.5 Chức năng tiện ích

Hàm tiện ích là các hàm đa năng thực hiện các tác vụ phổ biến hoặc cơ bản cần thiết cho việc phát triển ứng dụng. Node.js cung cấp một số hàm tiện ích được đóng gói trong các module tiện ích khác nhau như minh họa trong Hình 3.19.



Hình 3.19: Các mô-đun tiện ích

3.5.1 Làm việc với Mô-đun OS

Bảng 3.4 liệt kê các phương pháp của mô-đun hệ điều hành.

Tên của Phương pháp		Đặc trưng
loại hệ điều hành	Trả về tên của hệ điều hành	
os.platform	Trả về nền tảng của hệ điều hành	
os.totalmem	Trả về kích thước của tổng bộ nhớ hệ thống tính bằng byte	

Bảng 3.4: Các phương pháp của Mô-đun OS

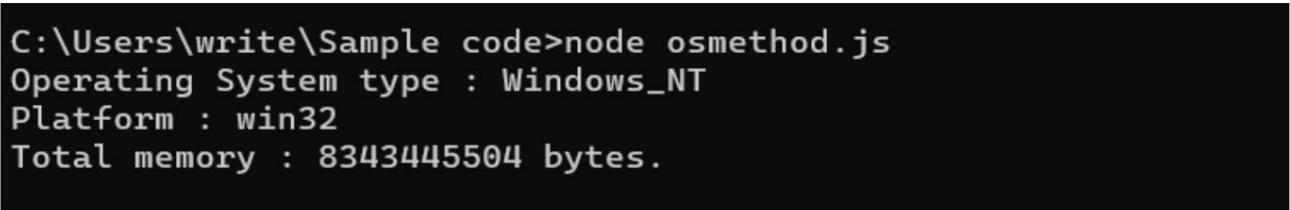
Đoạn mã 7 hiển thị mã để in tên hệ điều hành, nền tảng và tổng dung lượng bộ nhớ hệ thống tính bằng byte bằng cách sử dụng các phương thức của mô-đun hệ điều hành.

Đoạn mã 7:

```
const os = yêu cầu('os');
console.log('Loại hệ điều hành: + os.type()); console.log('Nền tảng: + os.platform());
console.log('Tổng bộ nhớ: + os.totalmem() + byte.');
```

Lưu mã vào tệp có tên osmethod.js và chạy tệp đó.

Hình 3.20 hiển thị đầu ra của đoạn mã này.



Hình 3.20: Thực thi osmethod.js

3.5.2 Làm việc với Mô-đun Đường dẫn

Bảng 3.5 liệt kê các phương thức của mô-đun Path.

Tên của phương pháp	Đặc trưng
đường dẫn.join(đường dẫn1, đường dẫn2,..)	Nối các chuỗi đã cho và tạo một chuỗi mới có thể được sử dụng làm thư mục hoặc đường dẫn
path.parse(đường dẫn)	Trả về một đối tượng cung cấp các chi tiết liên quan đến đường dẫn như gốc, thư mục, tên tệp và phần mở rộng tệp
path.dirname(đường dẫn)	Trả về các thư mục theo đường dẫn đã cho
path.extname(tên tệp hoặc đường dẫn)	Trả về phần mở rộng của một tập tin hoặc tệp trong đường dẫn

Bảng 3.5: Các phương pháp của Mô-đun Đường dẫn

Đoạn mã 8 hiển thị mã sử dụng mô-đun Path. Mã này sử dụng phương thức path.parse để hiển thị thông tin về gốc, cơ sở, thư mục, tên tệp và phần mở rộng của các đường dẫn /nodejs/sample code/sampleread.txt và /nodejs/sampleread.js.

tài liệu

Đoạn mã 8:

```
const path = require('path'); path1 =  
path.parse('/nodejs/mã mẫu/sampleread.txt');  
  
console.log(đường dẫn1);  
đường dẫn2 = đường dẫn.parse('/nodejs/sampleread.js');  
console.log(đường dẫn2);
```

Hình 3.21 hiển thị kết quả thực thi của Đoạn mã 8.

```
C:\Users\write\Sample code>node pathmethod.js  
{  
  root: '/',  
  dir: '/nodejs/sample code',  
  base: 'sampleread.txt',  
  ext: '.txt',  
  name: 'sampleread'  
}  
{  
  root: '/',  
  dir: '/nodejs',  
  base: 'sampleread.js',  
  ext: '.js',  
  name: 'sampleread'  
}
```

Hình 3.21: Thông tin về các tập tin

3.6 Tóm tắt

Mô-đun là tập hợp các hàm đã sẵn sàng để sử dụng.

Ba loại mô-đun là mô-đun cốt lõi, mô-đun cục bộ và mô-đun của bên thứ ba các mô-đun.

Các mô-đun lõi là các mô-đun tích hợp đi kèm với việc cài đặt

Node.js.

Gói là tập hợp một hoặc nhiều mô-đun được cài đặt bằng công cụ NPM.

Khung là tập hợp các thư viện và công cụ giúp tăng tốc phát triển ứng dụng.

Các hàm tiện ích thực hiện các tác vụ đa năng có thể hữu ích cho việc giao tiếp với hệ điều hành và làm việc với các tệp và thư mục.

Kiểm tra kiến thức của bạn

1. Bạn là một nhà phát triển web tại một công ty được phẩm. Bạn sẽ sử dụng mô-đun nào để tạo máy chủ web xử lý các yêu cầu HTTP?

- a) fs
- b) http c) hệ điều hành
- d) Không có câu nào trong số này

2. Trong các mô-đun sau đây, mô-đun nào là mô-đun tích hợp sẵn trong Node.js?

- a) khăng định b) chuỗi truy vấn
- c) thẻ hiển
- d) cây mangut

3. Đầu ra của đoạn mã đã cho sẽ là gì?

```
var_fs=yêu cầu('fs');  
console.log('Gói là tập hợp các mô-đun');
```

- a) Không hiển thị gì cả
- b) Hiển thị mô-đun fs không được nhận dạng
- c) Hiển thị lỗi trong mã
- d) Hiển thị các gói là một tập hợp các mô-đun

4. Bạn đang làm việc trên một dự án quản lý tệp với tư cách là trưởng nhóm phát triển. Bạn muốn kiểm tra xem một tệp đã tồn tại chưa và ghi đè dữ liệu trong đó. Nếu tệp chưa tồn tại, bạn muốn tạo một tệp và ghi dữ liệu vào đó. Bạn sẽ sử dụng phương pháp nào để thực hiện tình huống được đưa ra?

- a) fs.writeFile
- b) fs.appendFile
- c) fs.open
- d) Không có câu nào trong số này

5. Bạn cần giao tiếp với máy chủ từ xa để lấy và gửi dữ liệu. Bạn sẽ chọn framework nào để viết mã mà không phải lo lắng về vấn đề mạng?

- a) Khung API REST
- b) Khung MVC
- c) Khung MVC đầy đủ
- d) Khung khác

Câu trả lời để kiểm tra kiến thức của bạn

1	b
2	a, b 3
d 4	a
5	gió sáng

Hãy tự mình thử nhé

1. Bạn đang làm việc trên một ứng dụng quản lý tệp quản trị. Ứng dụng của bạn phải tạo hồ sơ quản trị, cập nhật thông tin quản trị, đọc dữ liệu quản trị, thêm thông tin mới và xóa hồ sơ quản trị. Hãy viết một chương trình sử dụng mô-đun hệ thống tệp.
 - a) Tạo một tệp có tên Admin.txt và ghi nội dung hồ sơ quản trị viên đã tạo thành công vào đó.
 - b) Tạo một tệp có tên adminOpenFile.txt với tệp văn bản Admin đã mở thành công. Sử dụng tập lệnh Node.js, mở tệp văn bản này, đọc và hiển thị nội dung của nó.
 - c) Ngoài ra, trong tập lệnh, hãy thêm văn bản Dữ liệu quản trị được thêm vào tệp Đã quản trị thành công tệp adminOpenFile.txt. mở
Đọc và hiển thị nội dung của tệp.
 - d) Thêm mã vào tập lệnh để xóa adminOpenFile.txt và hiển thị thông báo Tệp quản trị đã được xóa thành công.
2. Viết chương trình hiển thị hệ điều hành, nền tảng của hệ điều hành và kích thước tổng bộ nhớ hệ thống tính theo byte.
3. Viết chương trình hiển thị tất cả các thư mục trong một đường dẫn cụ thể.
4. Viết chương trình hiển thị phiên bản npm được cài đặt trong hệ thống của bạn.